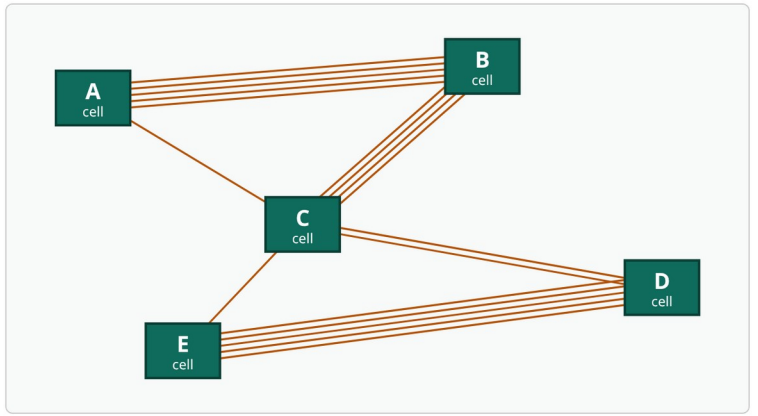
Initial bipartition

Before a computer can improve how a chip is split, it needs a first split

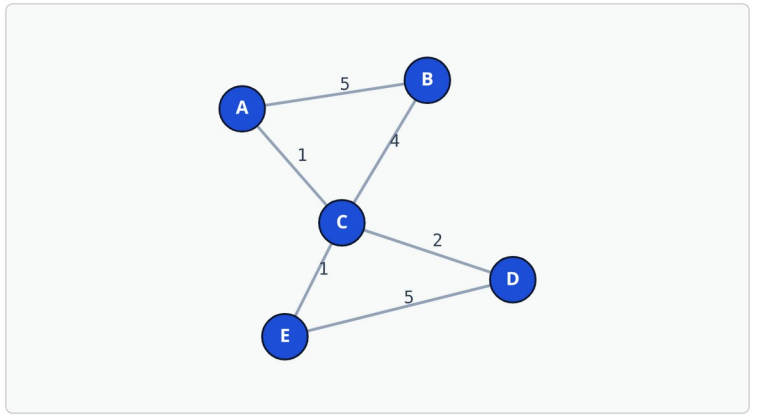
From a chip to a graph

- A chip is made of small building blocks called cells, joined by wires
- We draw each cell as a dot, called a node
- A line between two nodes is an edge: it means the two cells are wired together
- The number on an edge is its weight: how many wires join the pair
- Our example has five cells, A to E
- The heaviest edges, A–B and D–E, have five wires each

On the chip: cells joined by wires



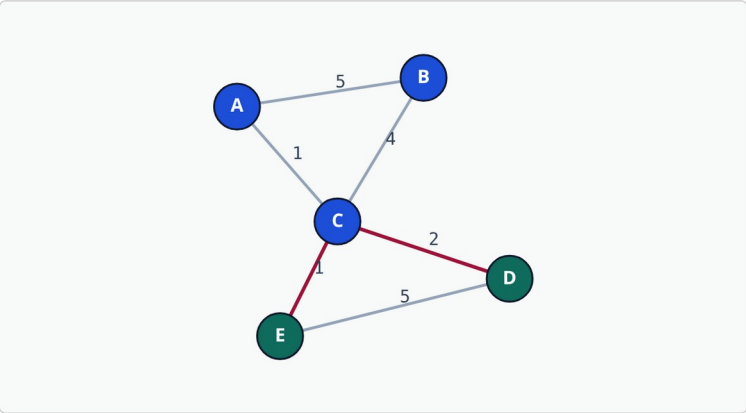
As a graph: weight = number of wires



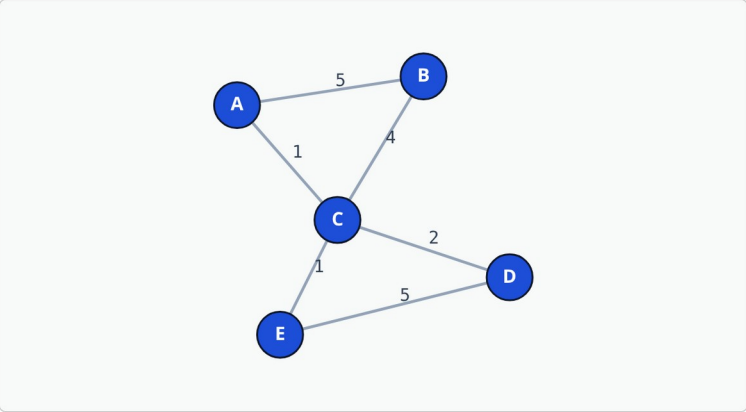
Two numbers: cut and balance

- Splitting a chip into two sides is called a bipartition
- Edges that cross between the two sides are cut, and they are drawn in red
- The cutsize adds up their weights: here, two plus one makes three
- Putting every cell on one side gives a cut of zero, but that split is useless
- So we also check balance: how many cells each side holds
- A split is legal when no side holds more than three of the five cells

A B C | D E: cut C–D 2 + C–E 1 = 3. Sizes 3 and 2: legal.



All five on one side: cut 0. Sizes 5 and 0: not legal.



Why a starting split?

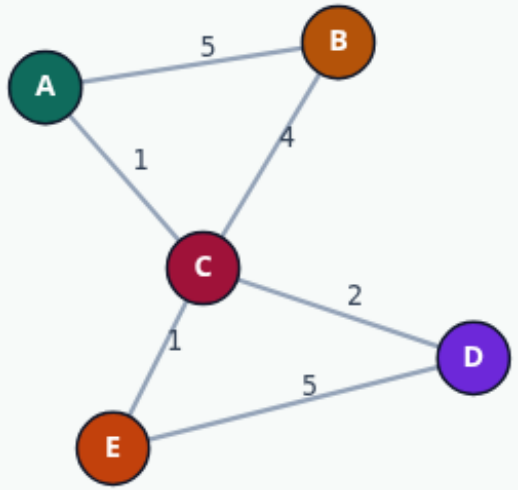
- Labs four and five teach two famous methods, called KL and FM
- They improve a split by moving cells from one side to the other
- But they can only improve: they need a legal split to start from
- Think of picking two teams for a game
- First you need any two teams, then you swap players to make the game fair
- A good first split leaves less swapping to do

Need a legal starting split

INITIAL BIPARTITION

Need a legal starting split

Step 1 / 5 · empty · module01-03-initial-bipartition



Explanation

Kernighan–Lin and FM only improve a split, so they need a first one. Every node goes on one of two sides, and no side may hold more than 3 of the 5 nodes.

- Every node on exactly one side
- Here: sides of 2 and 3 nodes
- Report cutsize and balance

```
5 nodes, 6 edges, total weight 18
legal sizes: 2 | 3
```

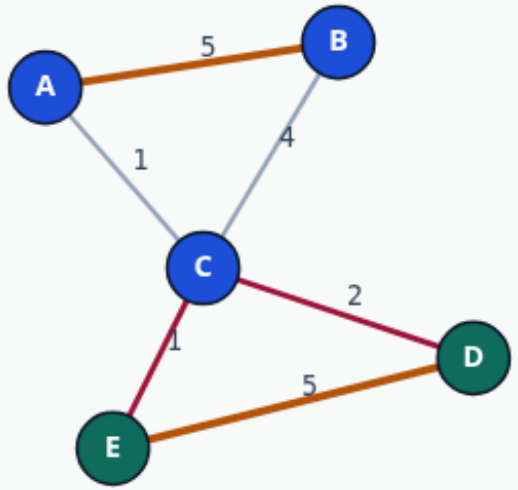
Need a legal starting split

Random seed 5: lucky ABC|DE cut 3

INITIAL BIPARTITION

Random seed 5: lucky ABC|DE cut 3

Step 2 / 5 · random-lucky · module01-03-initial-bipartition



Explanation

Random shuffles the nodes like cards and deals two of them to one side. Seed 5 deals D and E: only C–D and C–E are cut, cutsize 3. Seed 3 deals A, C, E: cutsize 16.

- Same seed, same split every time
- Seed 5 → ABC|DE, cut 3
- Seed 3 → ACE|BD, cut 16

```
method: random seed=5
parts: ABC|DE
cutsize: 3 = C–D 2 + C–E 1
```

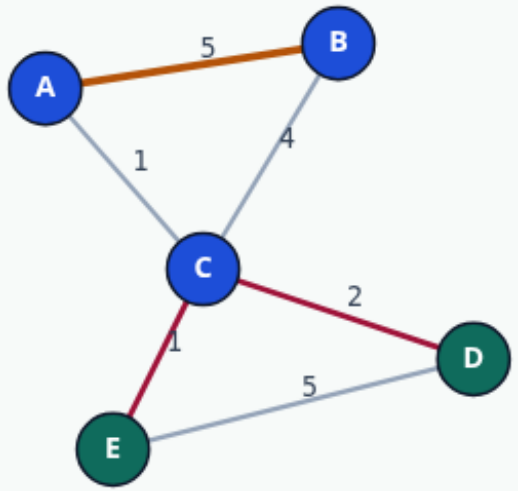
Random seed 5: lucky ABC|DE cut 3

Greedy: ABC|DE cut 3

INITIAL BIPARTITION

Greedy: ABC|DE cut 3

Step 3 / 5 · greedy · module01-03-initial-bipartition



Explanation

Greedy keeps the heaviest edge A–B inside one side (it ties with D–E; the smaller name wins). D, the node least tied to A and B, starts the other side. C joins A and B; E joins D, since that cuts 1 instead of 5.

- Anchor: A–B, weight 5
- Other side starts with D
- Each node goes where it adds less cut

```
method: greedy
parts: ABC|DE
cutsize: 3
```

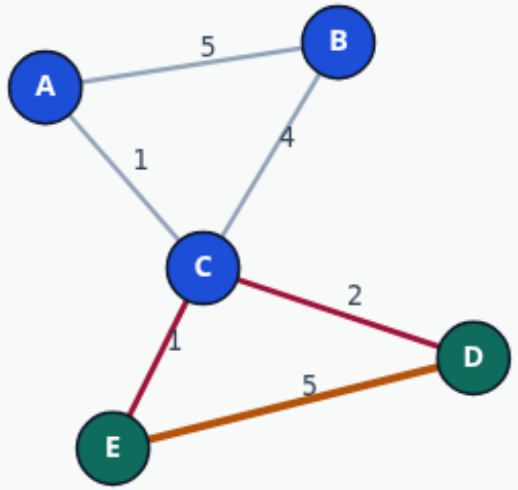
Greedy: ABC|DE cut 3

Grow from D: ABC|DE cut 3

INITIAL BIPARTITION

Grow from D: ABC|DE cut 3

Step 4 / 5 · grow-d · module01-03-initial-bipartition



Explanation

Grow starts a side with D and adds the node behind the heaviest edge leaving it: E, weight 5. The side now holds 2 of the 5 nodes, so it stops. A, B and C are the other side: cut 3.

- Start node: D
- Heaviest edge out: D–E (5)
- Grow from A → AB|CDE, cut 5

```
method: grow from D
parts: ABC|DE
cutsize: 3
```

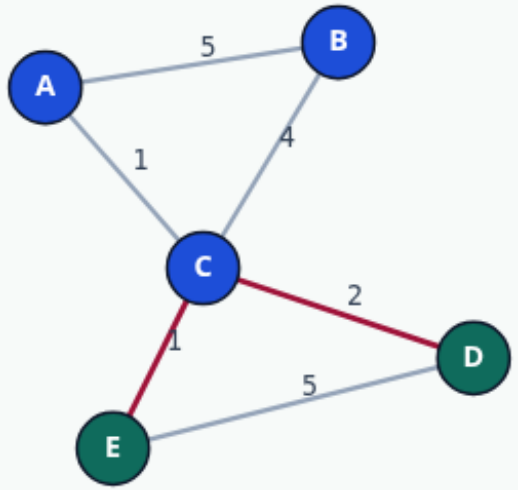
Grow from D: ABC|DE cut 3

Try several starts, keep the best

INITIAL BIPARTITION

Try several starts, keep the best

Step 5 / 5 · takeaway · module01-03-initial-bipartition



Explanation

Every start here is legal, but the cuts go from 3 to 16 on the same graph. Build several starts, drop any illegal one, and keep the smallest cut.

- Random: 3 to 16, depends on the seed
- Grow: 3, 5 or 9, depends on the start
- Greedy: 3 here, but not always

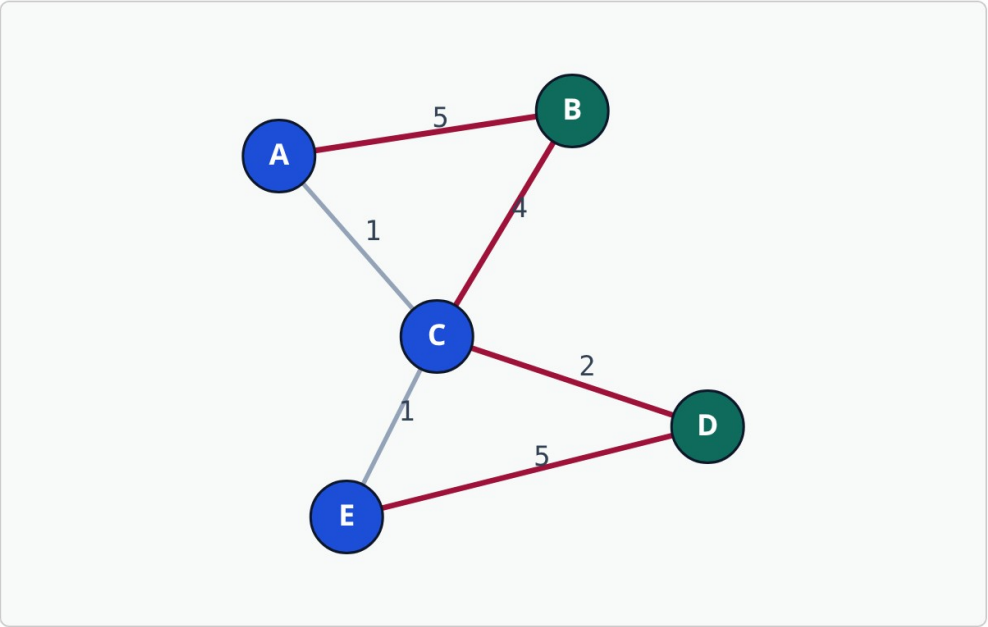
```
best: ABC|DE, cut 3
greedy 3 · grow A 5 · grow C 9 · grow D 3
```

Try several starts, keep the best

Worked example: an unlucky seed

- Now let's see how bad luck can get
- With seed three, the shuffle deals B and D to one side
- A, C and E end up on the other side
- Let's add up the cut, edge by edge
- A–B five, B–C four, C–D two and D–E five: sixteen in total
- Both heavy edges are cut, so this is the worst legal split of all

Random seed 3: A C E | B D

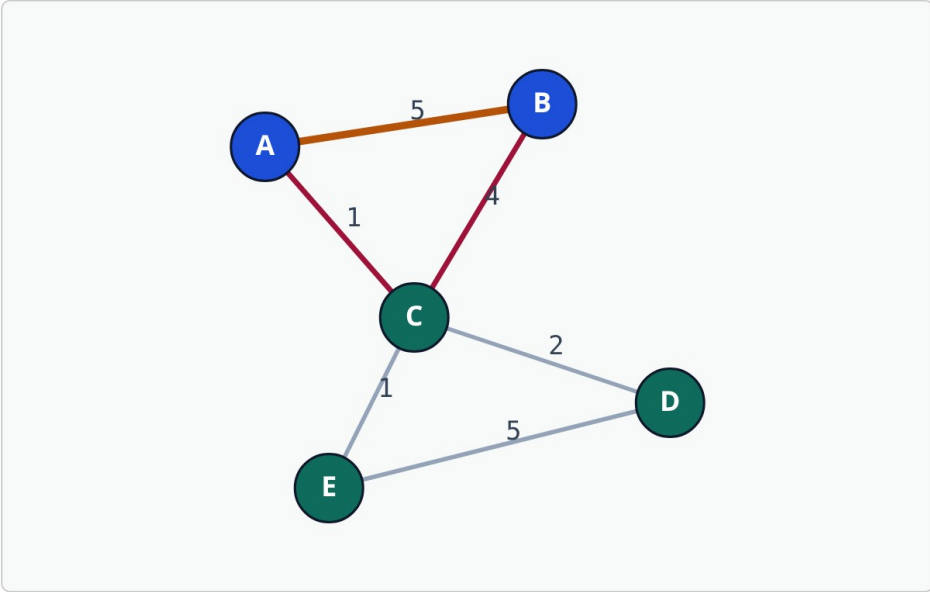


1. Cut (red): A–B, B–C, C–D, D–E
2. Cut: 5 + 4 + 2 + 5 = 16
3. **Sizes 3 and 2: legal, but the worst start**

Grow depends on the start node

- Grow has no luck in it, but it has a choice: the start node
- Start from A instead of D
- The heaviest edge out of A goes to B, with weight five, so B joins A
- With two cells, the side is full, and it stops
- Now C, D and E form the other side, so A–C and B–C are cut
- One plus four makes five, which is worse than three

Grow from A




```
1. Start the side with A
2. Heaviest edge out of A: A–B 5, add B
3. 2 of 5 nodes: the side is full, stop
4. Cut: A–C 1 + B–C 4 = 5
```

Keep the best legal start

- The fix is simple: never trust a single start
- Build several: a few random seeds, greedy, and grow from every node
- Throw away any split that breaks the size rule
- Keep the one with the smallest cut
- On our graph, greedy and the five grows give three, five, five, nine, three and three
- The best is three: A, B and C against D and E

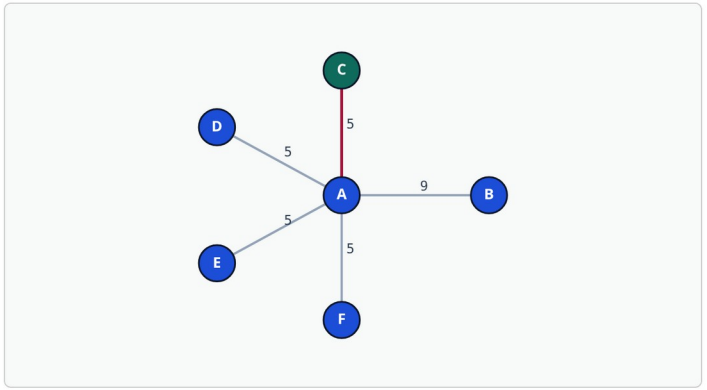
Keep the best legal start — try these

```
best_start(graph):
  starts = random(seed) for a few seeds
         + greedy
         + grow(x) for every node x
  drop every start that is not legal
  return the start with the smallest cutsize
```

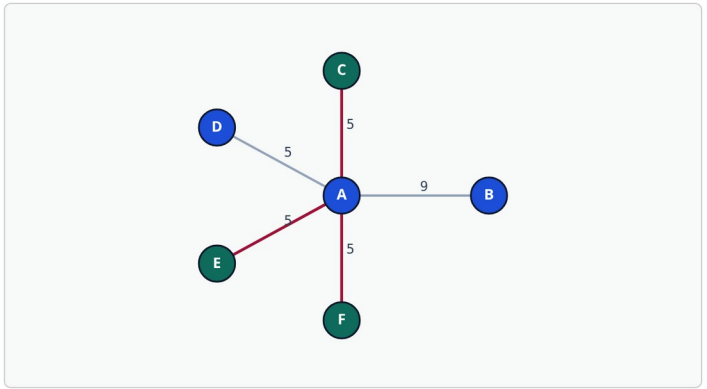
A size limit keeps the split legal

- Here is a star: A sits in the middle, wired to every other cell
- A–B weighs nine, and the four other edges weigh five
- With six cells, each side must hold exactly three
- An older greedy had no size limit, so it kept adding cells next to A
- It ended with five cells against one: cutsize five, but not legal
- Our greedy stops each side at three cells, and gets fifteen

Old greedy, no size limit: A B D E F | C. Sizes 5 and 1, cut 5. Not legal.



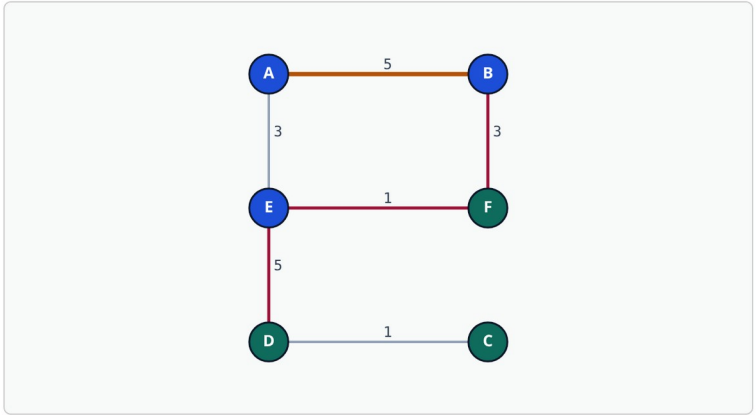
Greedy with the limit: A B D | C E F. Sizes 3 and 3, cut 15. Legal.



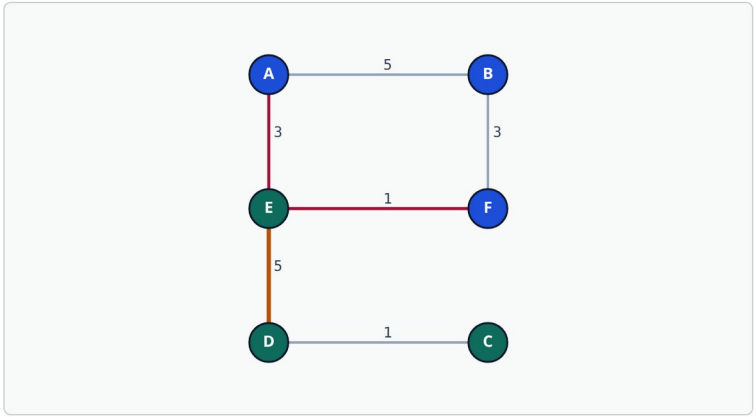
A tie can decide the answer

- In this graph, A–B and D–E both weigh five again
- Greedy must pick one of them to start from
- The rule picks the smaller name, A–B, and that gives a cutsize of nine
- Starting from D–E instead gives a cutsize of four
- So a simple tie rule more than doubled the cut
- Grow from C also finds four, which is one more reason to try several starts

Anchor A–B (the rule): A B E | C D F, cut 9



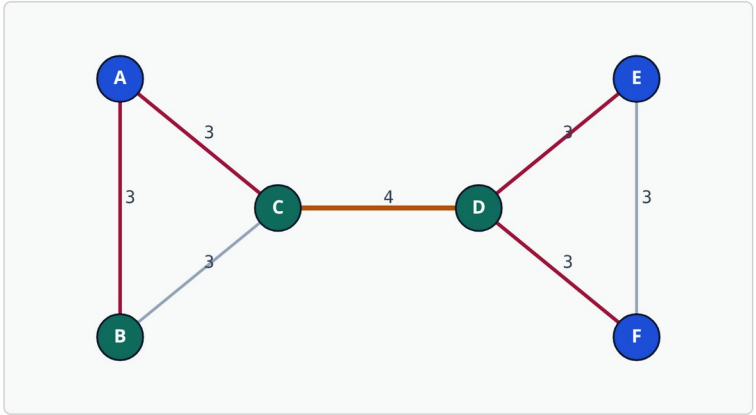
Anchor D–E: A B F | C D E, cut 4



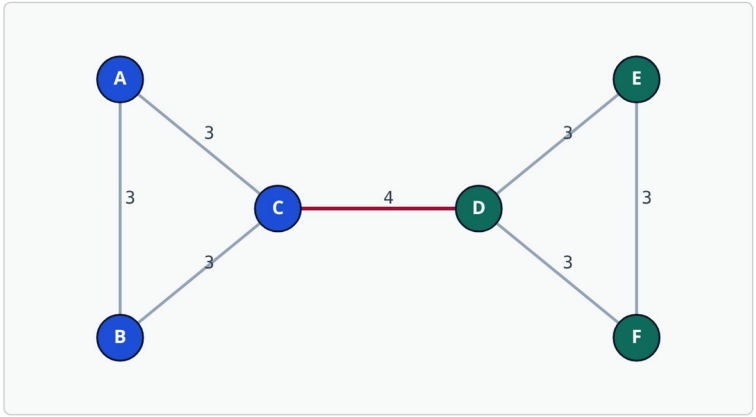
The heaviest wire is not always inside

- Here are two triangles, joined by one bridge, C–D, with weight four
- Every triangle edge weighs three
- The natural split puts one triangle on each side, and cuts only the bridge
- But greedy starts from the heaviest edge, and here that is the bridge itself
- So C and D begin on the same side, and the cut ends up at twelve
- Grow from A keeps its triangle together, and cuts only the bridge: four

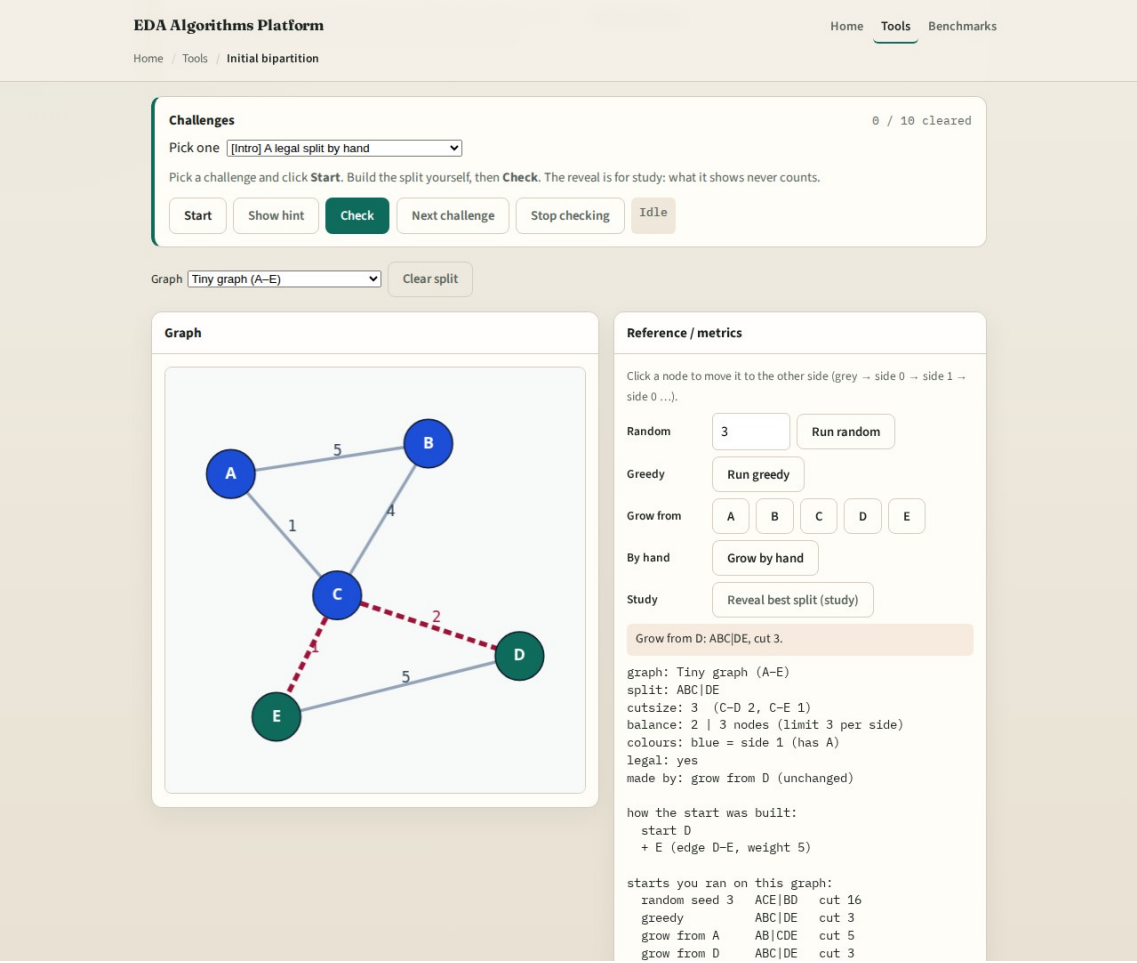
Greedy anchors the bridge C–D: A E F | B C D, cut 12



Grow from A: A B C | D E F, cut 4



Browser lab track



Browser lab starter

Implement track

wsl — module01-03-initial-bipartition/examples/.

```
# Track A — real Unix
# real shell session (Track A)

$ python3 initial_bipartition_check.py | grep PASS | head -14
PASS  tiny graph, random: seeds 0-15 give the golden splits and cuts
PASS  tiny graph, random: node order does not matter (input reversed)
PASS  tiny graph, random: every seed gives a legal 2|3 split
PASS  tiny graph, greedy: ABC|DE cut 3
PASS  star graph, greedy: ABD|CEF cut 15
PASS  tie graph, greedy: ABE|CDF cut 9
PASS  bridge graph, greedy: AEF|BCD cut 12
PASS  tiny graph, greedy: input reversed gives the same split
PASS  tie graph, greedy: input reversed gives the same split
PASS  star graph, greedy: both sides stay within ceil(n/2) nodes (3|3)
PASS  tiny graph, grow: from each node gives the golden split and cut
PASS  bridge graph, grow: from each node gives the golden split and cut
PASS  tiny graph, grow: input reversed gives the same splits
PASS  tiny graph, legality: everything on one side (cut 0) is NOT legal

$ python3 initial_bipartition_check.py | tail -1
all goldens match
```

Real shell session

Implement track — try these

```
# print every correct answer
python3 examples/initial_bipartition_check.py --show

# check your own functions
python3 examples/initial_bipartition_check.py --impl my_init.py
```

Pitfalls to watch

- Watch out for four common mistakes
- Reporting the cutsize without the balance
- Letting one side grow past its limit, which makes the cut look better than it really is
- Trusting a single random seed: on our graph, the cut ranged from three to sixteen
- And breaking ties by the order of the input, so the answer changes when the file changes

Recap: three ways to start

- Methods like KL and FM need a legal first split
- Random shuffles and deals: it is fast, but it depends on luck
- Greedy keeps the heaviest edge inside, then puts each cell where it cuts less
- Grow spreads from a start node, so the start node matters
- Ties and size limits can change the answer
- So build several starts, keep only the legal ones, and pick the smallest cut

Your turn

- Now it's your turn
- Pick a track: the browser lab, the Python track, or both
- Tick each box of the checklist as you go
- Then take the short quiz to check the main ideas
- Next comes Kernighan–Lin, which starts from your split and makes it better